

Trabalho 2º Trimestre — Relatório

Aluno(a): Pedro Otávio Adorna Rossato

Turma: 1F

Tema escolhido: Jogo Campo Minado

1. Pesquisa e fontes consultadas:

Foram consultados tutoriais do canal do YouTube Curso em Vídeo sobre manipulação de listas, matrizes e geração de números aleatórios com o módulo random, ministrado pelo professor Gustavo Guanabara (GUANABARA, 2017).

Principais vídeos utilizados: Curso de Python #18 - Listas (Parte 2); Exercício 86 - Matriz em Python; Exercício 87 - Mais sobre Matriz; Exercício 28 - Jogo da Adivinhação.

2. Lógica principal:

2.1 Representação do tabuleiro

O tabuleiro do modo clássico é representado por uma matriz 5x5 (lista de listas), em que cada célula inicia com o caractere "*", indicando que essa casa ainda não foi revelada. Quando for escolhida pelo jogador, a célula passa a armazenar o número de minas adjacentes calculado para aquela posição.

2.2 Distribuição das minas

As posições das duas minas são sorteadas como pares (linha, coluna) usando random.randint, com um laço while que sorteia novamente a segunda mina enquanto ela coincidir com a primeira, garantindo que ocupem posições distintas.

2.3 Contagem de minas adjacentes

A função conta_minas_adjacentes percorre as oito posições ao redor da célula escolhida, usando dois laços aninhados com deslocamentos de -1 a 1 nas direções de linha e coluna, ignorando a própria célula (deslocamento 0,0). Para cada posição vizinha válida dentro dos limites do tabuleiro, verifica-se se ela corresponde à posição de uma das minas, incrementando o contador correspondente.

2.4 Condição de vitória e derrota

No modo clássico, o jogador perde imediatamente ao escolher uma casa que contém uma mina, encerrando a partida. A vitória ocorre quando o jogador consegue revelar, com sucesso, todas as 23 casas do tabuleiro que não contém minas (as 25 casas da matriz 5x5, descontando as duas com bombas), contador controlado pela variável rodadas dentro do laço principal de jogo.

2.5 Modo Sobrevivência (funcionalidade extra)

Como funcionalidade adicional de criatividade, foi implementado um segundo modo de jogo, inspirado no Campo Minado, em que o tabuleiro é uma matriz 5x5 preenchida com números aleatórios de 0 a 9 (e não mais posições ocultas). O jogador possui um número de vidas (iniciando em 3, com limite máximo configurado como 6) e perde uma vida ao escolher uma casa cujo número sorteado coincide com um dos dois números definidos como "bomba" na rodada. Também há a possibilidade de ganhar uma vida extra ao escolher uma casa cujo número coincida com o número sorteado como vida extra, respeitando o limite máximo de vidas definido. O jogo termina quando as vidas chegam a zero, sendo exibida a quantidade de rodadas sobrevividas.

3. Organização do código e explicação das funções:

O código foi dividido em dez funções bem definidas, cada uma com uma responsabilidade específica, além do bloco main (if __name__ == '__main__':) como ponto de entrada. Essa separação facilita a leitura, a manutenção e o teste isolado de cada parte do programa:

- **int_classico():** exibe na tela as regras e instruções do modo Clássico, chamada a partir do menu quando o jogador escolhe a opção correspondente.
- **int_sobrevivencia():** exibe as regras e instruções do modo Sobrevivência, seguindo a mesma lógica da função anterior, porém para o modo extra.
- **conta_minas_adjacentes(linha, coluna, bomba1, bomba2, tabuleiro):** recebe a posição escolhida e as posições das duas minas e, usando dois laços aninhados (de -1 a 1), verifica cada uma das oito células vizinhas dentro dos limites do tabuleiro, retornando quantas delas correspondem a uma mina.
- **bomba_classico():** sorteia as coordenadas (linha, coluna) das duas minas do modo Clássico, repetindo o sorteio da segunda mina enquanto ela coincidir com a primeira, e retorna as duas posições.

- **mostrar_tabuleiro(tabuleiro):** percorre a matriz do modo Clássico e imprime o tabuleiro formatado no terminal, com os índices de coluna no topo e de linha à esquerda, alinhando cada célula para manter a organização visual.
- **jogar_classico():** função principal do modo Clássico. Cria o tabuleiro inicial, sorteia as minas com `bomba_classico()`, e controla o laço do jogo: exibe o tabuleiro, lê e valida a linha/coluna digitadas, verifica se a casa já foi aberta, chama `conta_minas_adjacentes()` para preencher a célula, atualiza o contador de rodadas e trata a derrota (ao acertar uma mina) e a vitória (ao completar as 23 casas seguras).
- **bomba_sobrevivencia():** sorteia os dois números tratados como "bomba" e o número que concede vida extra no modo Sobrevivência, retornando os três valores para uso em `jogar_sobrevivencia()`.
- **tab_sobrevivencia():** gera e retorna uma matriz 5x5 preenchida com números aleatórios de 0 a 9, que serve como tabuleiro do modo Sobrevivência.
- **jogar_sobrevivencia():** função principal do modo Sobrevivência. Gera o tabuleiro e os números especiais, e controla o laço do jogo enquanto houver vidas: lê e valida a linha/coluna, exibe o número escolhido, verifica se ele é uma bomba, o número de vida extra ou um número neutro, atualiza vidas e rodadas de acordo, e encerra o jogo exibindo o total de rodadas sobrevividas quando as vidas chegam a zero.
- **menu():** exibe a arte em ASCII e o menu principal em um laço contínuo, lê a opção digitada pelo jogador e direciona para `jogar_classico()`, `jogar_sobrevivencia()`, as funções de instrução, ou encerra o programa na opção de saída.

4. Principais problemas encontrados e soluções:

1. Validação de entrada do usuário: entradas não numéricas ou fora do intervalo do tabuleiro causavam erros. Solução foi usar os `isdigit()` para validar se a entrada é numérica antes da conversão, além de verificação dos limites válidos (0 a 4).
2. Sobreposição de posições de minas: havia risco de as duas minas serem sorteadas na mesma posição. Solução foi o uso de um laço `while` que sorteia novamente a segunda mina enquanto ela coincidir com a primeira.
3. Formação do tabuleiro do modo Clássico: ao imprimir o tabuleiro, os valores ficavam desalinhados entre as linhas e as colunas, dificultando a leitura. A solução foi colocar uma largura fixa (`:%5s`) para cada célula em um espaço de tamanho constante.

5. Declaração de uso de Inteligência Artificial:

Durante o desenvolvimento, foram utilizadas ferramentas de Inteligência Artificial. Seu uso teve como objetivo revisar o código já escrito e identificar pontos de melhoria (como a arte em ASCII do menu, para deixar mais atrativo, e a forma de implementar o sistema de contagem de minas adjacentes e como estruturar melhor o tabuleiro). A lógica principal, a escrita do código e as decisões sobre os aspectos de criatividade do jogo (como a criação do modo sobrevivência) foram desenvolvidas pelo aluno.

6. Principais referências:

GUANABARA, Gustavo. Curso de Python #18 - Listas (Parte 2). Curso em Vídeo (YouTube), 2017.

Disponível em: https://www.youtube.com/watch?v=YV_JQmZNFsk&t=481s. Acesso em: 11 jul. 2026.

GUANABARA, Gustavo. Exercício 86 - Matriz em Python. Curso em Vídeo (YouTube), 2017. Disponível em: <https://www.youtube.com/watch?v=EGmIFdwD4C4&t=508s>. Acesso em: 11 jul. 2026.

GUANABARA, Gustavo. Exercício 87 - Mais sobre Matriz. Curso em Vídeo (YouTube), 2017. Disponível em: <https://www.youtube.com/watch?v=QhS829x6up4>. Acesso em: 11 jul. 2026.

GUANABARA, Gustavo. Exercício 28 - Jogo da Adivinhação. Curso em Vídeo (YouTube), 2017. Disponível em: <https://www.youtube.com/watch?v=kchC5KLZSZ4>. Acesso em: 11 jul. 2026.